UNIVERSITI TEKNOLOGI MALAYSIA

INDUSTRIAL TRAINING REPORT

VLSI & EMBEDDED COMPUTING ARCHITECTURE DESIGN (VeCAD) RESEARCH LABORATORY

A RISC-V Custom-Instruction Accelerator for Fixed-Point Min-Sum LDPC Decoding on FPGA

Faculty of Electrical Engineering

Universiti Teknologi Malaysia, 81310 Johor Bahru, Johor, Malaysia

BACHELOR OF ENGINEERING (COMPUTER ENGINEERING)

ANAS TAREK GEBREEL

Faculty of Electrical Engineering

Universiti Teknologi Malaysia

Johor Bahru, Malaysia

October 2026

DECLARATION OF THE UNDERGRADUATE INDUSTRIAL TRAINING REPORT AND COPYRIGHT

Trainee full name	:	ANAS TAREK GEBREEL
Matric Number	:	A22EE0006
Date of Birth	:	30/07/2003
Title	:	INDUSTRIAL TRAINING REPORT
Academic Session	:	2025/2026
Date	:	8th OCTOBER 2026

I declare that this industrial training report is classified as:

	CONFIDENTIAL	(Contains confidential information under the Official Secret Act 1972)*
	RESTRICTED	(Contains restricted information as specified by the organization where research was done)*
✓	**INTERNAL**	I agree that the industrial training report is for internal use only

I acknowledge that Universiti Teknologi Malaysia reserves the right as follows:

1. The industrial training report is the property of Universiti Teknologi Malaysia.
2. The Faculty of Electrical Engineering has the right to make copies for the purpose of reference only.

Certified by:	**Verified by:**
..... INDUSTRY SUPERVISOR SIGNATURE Name: Dr. Jeevan VeCAD Research Laboratory, UTM	..... FACULTY SUPERVISOR SIGNATURE AND STAMP Name: .....
Date: [XX] OCTOBER 2026	Date: [XX] OCTOBER 2026

Industrial Training Report Evaluation (FKE – LP6)

Technical Report Evaluation Rubric

Purpose: Evaluate a student's ability to write a technical report.

Ranking: On a scale from 1 (lowest performance) to 5 (highest performance), assign points to each dimension based on the criteria below.

Criteria	5	4	3	2	1	Score
1. Introduction	Directly relevant	Somewhat relevant	Remotely related	Totally unrelated	Totally unrelated	
2. Company Background	Information is presented in a logical, interesting way, which is easy to follow	Information is presented in a logical manner, which is easily followed	Work is hard to follow as there is very little continuity	Sequence of information is difficult to follow; no apparent structure or continuity	Sequence of information is difficult to follow; no apparent structure or continuity	
3. Provision of Training	Develops a detailed training schedule identifying and explaining ALL significant steps and activities, their interrelationships, and the resources responsible	Develops a training schedule identifying and explaining major steps and activities, their interrelationships, and the resources responsible	Develops a training schedule identifying major steps and activities but not their interrelationships or responsible resources	Develops an incomplete training schedule that neither identifies major steps/activities nor their interrelationships	Develops an incomplete training schedule that neither identifies major steps/activities nor their interrelationships	
4. Project	Thoroughly discusses all required items: business description, need addressed, project scope and feasibility, schedule, and a resource plan and budget	Discusses all required items, though with limited detail on some items	Identifies most of the required items but provides limited detail	Identifies few, if any, of the required items and provides insufficient detail	Identifies few, if any, of the required items and provides insufficient detail	
5. Conclusion	Presents a logical explanation for the findings and fully addresses the objectives	Presents a logical explanation for the findings and addresses the objectives	Presents a partly illogical explanation for the findings	Presents an illogical explanation for the findings	Presents an illogical explanation for the findings	
6. Writing / Mechanics	Sentences are well constructed; no grammar, spelling or punctuation errors	Sentences are well constructed; few errors	Sentences are adequately constructed; some errors	Sentences are poorly constructed; many errors	Sentences are poorly constructed; numerous errors	
TOTAL					(/ 30)	

I formally declare that this report has been read and evaluated.

. . . . . .

[FACULTY SUPERVISOR]

ACKNOWLEDGEMENT

First and foremost, I would like to express my deepest gratitude to my supervisor, Dr. Jeevan, of the VLSI & Embedded Computing Architecture Design (VeCAD) Research Laboratory at the Faculty of Electrical Engineering, Universiti Teknologi Malaysia, for his continuous guidance, patience, and encouragement throughout my industrial training. His technical insight into digital design, FPGA systems, and channel coding shaped both the direction and the quality of this project.

I am also thankful to the researchers, postgraduate students, and fellow interns at the VeCAD laboratory for the collaborative and supportive environment they provided. Their willingness to share knowledge and discuss ideas made the training both productive and enjoyable.

My appreciation extends to the Faculty of Electrical Engineering and the industrial training committee of Universiti Teknologi Malaysia for organising this training programme and giving me the opportunity to apply my academic knowledge in a real research setting. Finally, I would like to thank my family and friends for their constant support and motivation throughout this period.

ABSTRACT

This report documents the industrial training undertaken at the VLSI & Embedded Computing Architecture Design (VeCAD) Research Laboratory, Faculty of Electrical Engineering, Universiti Teknologi Malaysia. The training focused on digital hardware design and hardware/software co-design on a field-programmable gate array (FPGA) platform, progressing from bringing up soft-core processors on the Terasic DE2-115 board to designing a complete domain-specific hardware accelerator.

The special technical project was the design of a RISC-V custom-instruction accelerator for fixed-point min-sum decoding of low-density parity-check (LDPC) codes. A baseline min-sum decoder was first implemented in software on the open-source NEORV32 RISC-V soft core, then accelerated by extending the processor with a custom functional unit (CFU) performing the check-node min-sum operation in hardware. The design was synthesised, verified on the DE2-115 (Intel Cyclone IV E) and measured. The accelerator achieved an approximately five-fold reduction in the cycles per check-node update at a cost of only 128 logic elements (about 0.1% of the device), maintained a maximum operating frequency of 59.04 MHz, and introduced no loss of decoding accuracy relative to the software baseline. A fixed-point study showed that a 5-6 bit quantisation of the log-likelihood ratios matches a 32-bit floating-point reference. The report presents the company background, the conduct of the training including a detailed daily log, and the full theory, methodology, results and conclusions of the technical project.

ABSTRAK

Laporan ini mendokumentasikan latihan industri yang dijalankan di Makmal Penyelidikan Reka Bentuk Seni Bina Pengkomputeran VLSI & Terbenam (VeCAD), Fakulti Kejuruteraan Elektrik, Universiti Teknologi Malaysia. Latihan ini memberi tumpuan kepada reka bentuk perkakasan digital serta reka bentuk bersama perkakasan/perisian pada platform tatasusunan get boleh atur cara medan (FPGA), bermula daripada menghidupkan pemproses teras lembut pada papan Terasic DE2-115 sehingga mereka bentuk satu pemecut perkakasan khusus domain yang lengkap.

Projek teknikal khas ialah reka bentuk satu pemecut arahan tersuai berasaskan RISC-V untuk penyahkodan min-sum titik tetap bagi kod semakan pariti berketumpatan rendah (LDPC). Satu penyahkod min-sum asas mula-mula dilaksanakan dalam perisian pada teras lembut RISC-V NEORV32 sumber terbuka, kemudian dipercepatkan dengan melanjutkan pemproses menggunakan unit fungsi tersuai (CFU) yang melaksanakan operasi nod semakan min-sum dalam perkakasan. Reka bentuk ini disintesis, disahkan pada papan DE2-115 (Intel Cyclone IV E) dan diukur. Pemecut ini mencapai pengurangan kira-kira lima kali ganda dalam bilangan kitaran bagi setiap kemas kini nod semakan dengan kos hanya 128 elemen logik (kira-kira 0.1% daripada peranti), mengekalkan frekuensi operasi maksimum sebanyak 59.04 MHz, dan tidak menyebabkan sebarang kehilangan ketepatan penyahkodan berbanding asas perisian. Satu kajian titik tetap menunjukkan bahawa kuantisasi 5-6 bit bagi nisbah kebolehjadian-log sepadan dengan rujukan titik terapung 32-bit. Laporan ini membentangkan latar belakang organisasi, perjalanan latihan termasuk log harian terperinci, serta teori, metodologi, keputusan dan kesimpulan projek teknikal tersebut sepenuhnya.

TABLE OF CONTENTS

ACKNOWLEDGEMENT ..... **i**
ABSTRACT ..... **ii**
ABSTRAK ..... **iii**
TABLE OF CONTENTS ..... **iv**
LIST OF TABLES ..... **v**
LIST OF FIGURES ..... **vi**
LIST OF ABBREVIATIONS ..... **vii**
CHAPTER 1 INTRODUCTION **1**
1.1 Industrial Training 1
1.2 Company Background 2
1.3 Company Structure 3
1.4 Research and Business Model 3
1.5 Company Expertise 3
CHAPTER 2 CONDUCT OF INDUSTRIAL TRAINING **5**
2.1 Overview of the Training 5
2.2 Development Environment and Tools 5
2.3 Weekly Training Schedule 8
2.4 Detailed Daily Log 8
2.5 Skills and Knowledge Acquired 14
CHAPTER 3 SPECIAL TECHNICAL PROJECT **15**
3.1 Project Background 15
3.2 Objectives 16
3.3 Theoretical Foundation 16
3.4 Methodology 18
3.5 Results and Discussion 22
3.6 Conclusion 27
CHAPTER 4 CONCLUSION **28**
REFERENCES **29**
APPENDIX **29**

LIST OF TABLES

TABLE NO.	TITLE
Table 2.1	Weekly industrial training schedule
Table 3.1	Fixed-point quantisation versus 32-bit floating-point reference
Table 3.2	Decoder latency: software versus custom-instruction accelerator
Table 3.3	FPGA resource utilisation and timing (Cyclone IV E)
Table 3.4	Comparison with the software baseline and a reference work

LIST OF FIGURES

FIGURE NO.	TITLE
Figure 2.1	Intel Quartus Prime Lite installation and device selection
Figure 2.2	Quartus Prime New Project Wizard
Figure 2.3	Terasic DE2-115 development board
Figure 2.4	NEORV32 RISC-V project in the Quartus Prime environment
Figure 3.1	Top-level HDL design in the Quartus text editor
Figure 3.2	Software min-sum reference decoder output (console)
Figure 3.3	Quartus Prime compilation flow (Tasks pane)
Figure 3.4	Quartus Timing Analyzer
Figure 3.5	Bit-error-rate performance at higher SNR
Figure 3.6	Bit-error-rate performance at lower SNR
Figure 3.7	Average number of decoding iterations versus SNR
Figure 3.8	Measured BER and average iterations (console output)
Figure 3.9	Clock cycles per check-node update: software vs CFU
Figure 3.10	Quartus Flow Summary (resource utilisation)
Figure 3.11	LDPC decoder running on the DE2-115 (board LEDs)

LIST OF ABBREVIATIONS

LDPC	-	Low-Density Parity-Check (code)
FPGA	-	Field-Programmable Gate Array
ASIC	-	Application-Specific Integrated Circuit
CFU	-	Custom Functions Unit
ISA	-	Instruction Set Architecture
RTL	-	Register-Transfer Level
LLR	-	Log-Likelihood Ratio
BER	-	Bit Error Rate
SNR	-	Signal-to-Noise Ratio
AWGN	-	Additive White Gaussian Noise
CNU / VNU	-	Check-Node Unit / Variable-Node Unit
LE	-	Logic Element
HDL	-	Hardware Description Language
SoC	-	System-on-Chip
SDRAM	-	Synchronous Dynamic Random-Access Memory
PLL	-	Phase-Locked Loop
UART	-	Universal Asynchronous Receiver/Transmitter
VeCAD	-	VLSI & Embedded Computing Architecture Design
UTM	-	Universiti Teknologi Malaysia

CHAPTER 1 INTRODUCTION

The field of computer engineering is evolving rapidly, with a continuous push towards higher performance, lower power consumption, and more specialised computing architectures. As the demand for complex digital systems grows across industries ranging from telecommunications and aerospace to artificial intelligence, the role of custom digital hardware becomes increasingly important. Industrial training is a fundamental component of the computer engineering curriculum, giving students the opportunity to bridge the gap between theoretical academic concepts and practical engineering challenges.

This report documents the industrial training carried out at the VLSI & Embedded Computing Architecture Design (VeCAD) Research Laboratory, within the Faculty of Electrical Engineering at Universiti Teknologi Malaysia (UTM) in Johor Bahru. During the training, the trainee was immersed in field-programmable gate array (FPGA) design, soft-processor integration, and the development of a custom hardware accelerator for an advanced digital-communication algorithm. The platform used throughout was the Terasic DE2-115 board, featuring an Intel Cyclone IV E FPGA.

1.1 Industrial Training

Industrial training is a compulsory component of the Bachelor of Engineering (Computer Engineering) programme at Universiti Teknologi Malaysia. Its purpose is to expose students to a professional working and research environment, allowing them to apply the theoretical knowledge gained during their studies to practical engineering problems, develop technical and interpersonal skills, and gain first-hand experience of the engineering design process.

This training was carried out at the VeCAD Research Laboratory over a period of approximately eleven weeks, beginning on 20 July 2026, under the supervision of Dr. Jeevan. The training was structured to build competencies systematically, starting with foundational system-on-chip concepts using the Nios II processor, progressing to the open-source RISC-V architecture, and culminating in a special technical project: a custom-instruction hardware accelerator for low-density parity-check (LDPC) decoding.

1.2 Company Background

The VeCAD Research Laboratory is a research group within the Faculty of Electrical Engineering at Universiti Teknologi Malaysia, located in Johor Bahru, Malaysia. It is dedicated to advancing custom digital hardware design, with a strong emphasis on verylarge-scale integration (VLSI), system-on-chip (SoC) architectures, and complex embedded systems targeting both application-specific integrated circuits (ASICs) and FPGAs. Rather than a commercial enterprise, VeCAD operates as an academic research organisation whose outputs are research results, prototype hardware, trained engineers, and publications.

VeCAD operates at the intersection of computer architecture, digital logic design, and applied mathematics. It is composed of academic staff, postgraduate researchers, and undergraduate interns, fostering a collaborative and highly technical environment. The laboratory is equipped with industry-standard electronic design automation (EDA) tools and FPGA development platforms, including the Terasic DE2-115 board based on the Intel Cyclone IV E device.

1.3 Company Structure

The VeCAD laboratory is headed by an academic staff member who serves as the laboratory head and is supported by a team of academic supervisors, postgraduate researchers at both master's and doctoral levels, and undergraduate trainees. Each project is guided by a supervising lecturer who provides technical direction and monitors progress. During this training, the trainee reported directly to Dr. Jeevan, who assigned the project objectives, provided weekly feedback, and reviewed the technical outcomes. The relatively flat structure of the laboratory encourages close interaction between supervisors, researchers, and trainees, and allows knowledge to be shared freely across projects.

1.4 Research and Business Model

As an academic research laboratory, VeCAD does not operate on a conventional commercial business model. Its activities are sustained through university funding, competitive research grants, and collaboration with industry and other institutions. The laboratory's value is delivered through the generation of new knowledge, the publication of research, the development of prototype hardware and intellectual property, and the training of skilled engineers who subsequently contribute to the semiconductor and electronics industry. Research undertaken in the laboratory frequently targets problems of practical relevance to the communications, computing, and embedded-systems sectors.

1.5 Company Expertise

The core expertise of the VeCAD laboratory lies in digital integrated-circuit and system design. A major focus is the hardware acceleration of computationally intensive algorithms, including custom accelerators for artificial intelligence and machine learning such as convolutional neural networks used in applications like real-time

driver-drowsiness detection. A second area is cryptography and secure hardware design, where the laboratory studies hardware implementations of cryptographic primitives that are resilient to side-channel attacks, as well as biometric authentication systems.

A third area, which formed the core of this internship, is digital-communications research, specifically advanced error-correction decoding. The design of efficient hardware decoders for codes such as LDPC and polar codes is essential to meeting the high-throughput and low-latency requirements of modern standards such as 5G, Wi-Fi, and next-generation satellite links. The laboratory's proficiency with the whole design cycle, from HDL modeling and synthesis to place-and-route, timing closure, and on-board verification, served as the technical basis for the unique technological endeavor detailed in this report.

CHAPTER 2 CONDUCT OF INDUSTRIAL TRAINING

2.1 Overview of the Training

The training was organised so that the trainee progressively built the skills required for the final technical project. The early weeks established the FPGA design flow and brought up soft-core processors; the middle weeks introduced custom-instruction design, memory and peripheral integration, multi-core systems, and the open-source RISC-V core; and the later weeks were devoted to the design, integration, measurement, and documentation of the LDPC custom-instruction accelerator. Each week concluded with a short review with the supervisor, and progress was recorded in a weekly logbook.

2.2 Development Environment and Tools

The trainee worked primarily with Intel Quartus Prime (versions 13.1 and Lite 25.1), the Platform Designer (Qsys) system-integration tool, the Nios II and NEORV32 soft processors, and the Terasic DE2-115 development board. The first weeks were spent installing and configuring this toolchain, including selecting the Cyclone IV E device support during the Quartus Prime Lite installation, as shown in Figure 2.1.

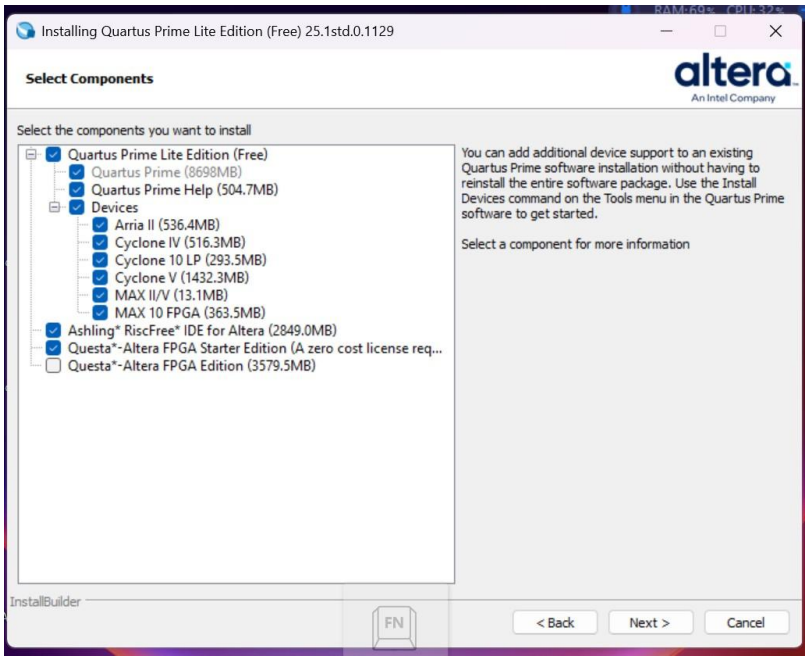


Figure 2.1 Intel Quartus Prime Lite installation and Cyclone IV E device selection.

Each hardware design began from a Quartus project created through the New Project Wizard, in which the working directory, project name, top-level entity, and target device (EP4CE115F29C7) were specified, as shown in Figure 2.2.

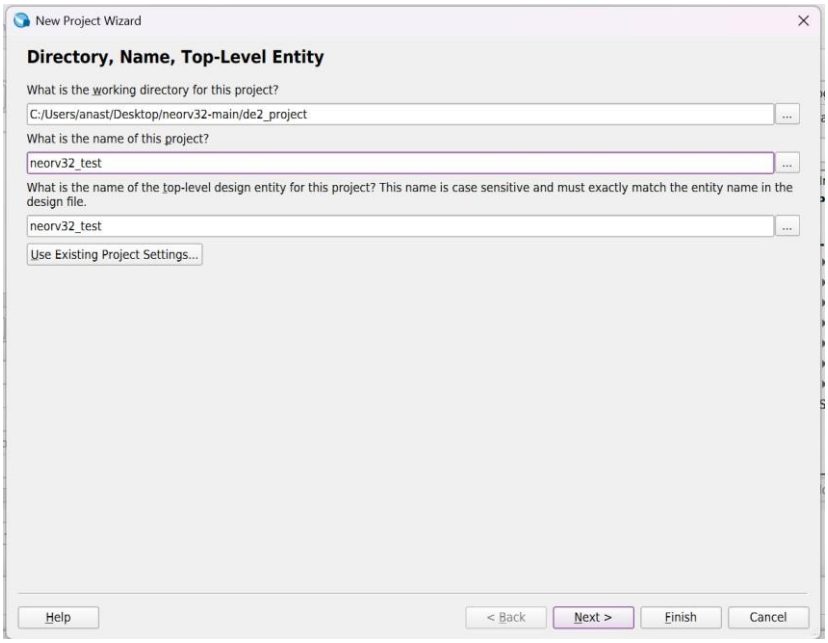


Figure 2.2 Quartus Prime New Project Wizard used to configure each hardware project.

All designs were synthesised for, and verified on, the Terasic DE2-115 development board shown in Figure 2.3. The board provides the Cyclone IV E FPGA together with 128 MB of SDRAM, red and green LEDs, seven-segment displays, switches, and pushbuttons, which were used extensively to display results during verification.

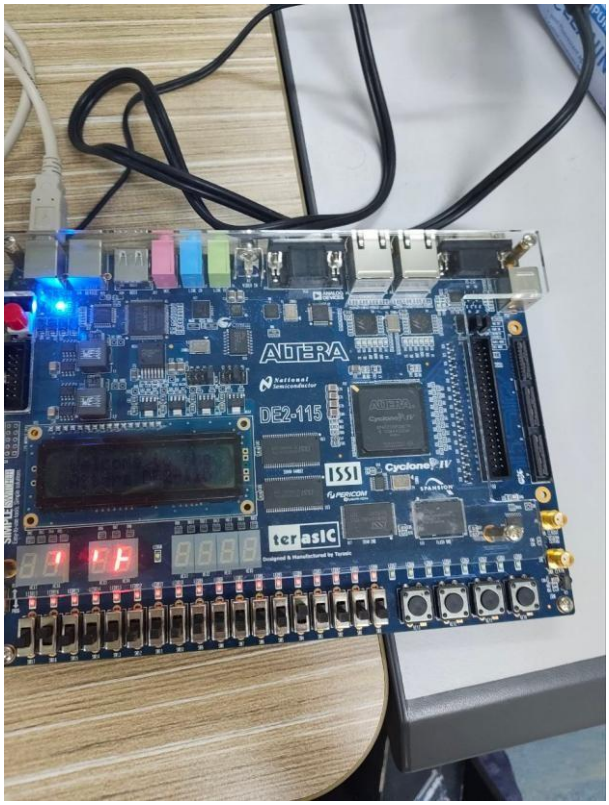


Figure 2.3 Terasic DE2-115 development board (Intel Cyclone IV E) used for all hardware implementations.

In the later part of the training the open-source NEORV32 RISC-V soft core was adopted as the host processor for the LDPC decoder. Figure 2.4 shows the NEORV32 project open in the Quartus Prime environment, with the processor source files, project navigator, and IP catalogue.

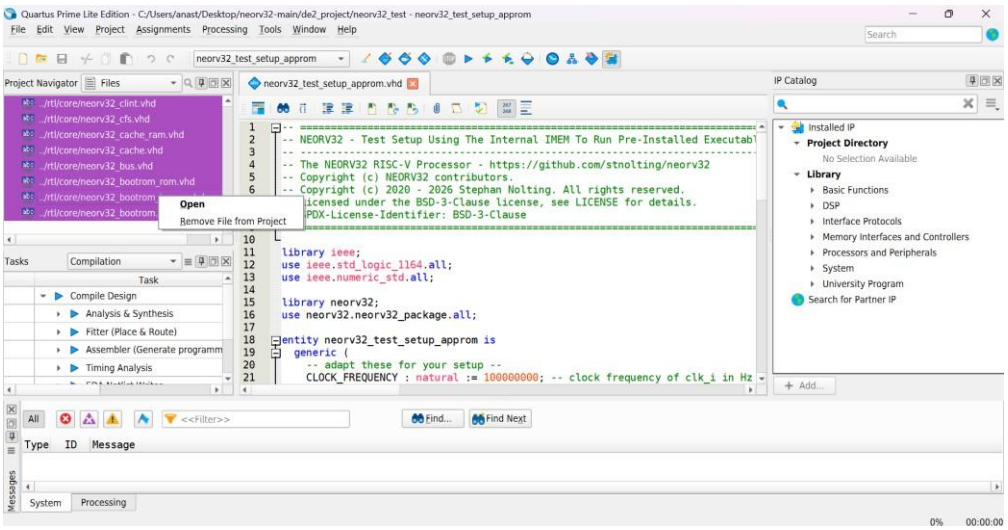


Figure 2.4 NEORV32 RISC-V project in the Quartus Prime environment.

2.3 Weekly Training Schedule

A summary of the weekly activities carried out during the training is presented in Table 2.1. A more detailed day-by-day account follows in Section 2.4. **Table 2.1 Weekly industrial training schedule**

WEEK	GENERAL TASK	DESCRIPTION
1	**FPGA environment & Nios II bring-up**	Installed and configured Quartus Prime and the Nios II toolchain; built a first Nios II/e system and ran a program on the DE2-115.
2	**Custom instruction (Nios II)**	Designed a hardware custom instruction in Verilog, connected it to the Nios II through Platform Designer, and invoked it from C.
3	**External memory & I/O**	Added a PLL and the 128 MB SDRAM controller; benchmarked SDRAM versus on-chip memory and drove the LEDs through a PIO peripheral.
4	**Multi-core system**	Built a dual-core Nios II system with per-core memory, mutexes, and vectors, running independent programs on each core.
5	**RISC-V (NEORV32) bring-up**	Migrated to the open-source NEORV32 RISC-V soft core; scripted the VHDL-2008 compilation and established the RISC-V build-and-run flow.
6	**LDPC decoder (SW & demo)**	Implemented a min-sum LDPC decoder in C, ported it to the NEORV32 core, and built an interactive LED demonstration.
7	**Characterisation**	Ran Monte-Carlo BER simulations and prepared a literature review and technical presentation on LDPC codes and minsum decoding.
8	**CFU design & quantisation study**	Designed the custom functional unit performing the checknode min-sum operation; carried out a fixed-point quantisation study (3-6 bit).
9	**CFU integration**	Integrated the CFU into the NEORV32 datapath via the RISCV_ISA_Xcfu interface and verified functional equivalence against the software decoder.
10	**Synthesis, verification & measurement**	Synthesised the CPU+CFU system, verified correctness on the DE2-115, and measured speedup, resource utilisation, and maximum frequency.
11	**Documentation & reporting**	Finalised the research paper, prepared the industrial training final report, and archived and handed over the project.

2.4 Detailed Daily Log

The following section provides a day-by-day account of the activities undertaken during the training, illustrating the progressive learning curve and the systematic execution of the project methodology.

Week 1: Platform Setup and First Processor

Day 1: Introduction to the VeCAD laboratory, safety briefing, and orientation on the lab's ongoing research. Received the Terasic DE2-115 board and installed Intel Quartus Prime and the Nios II Software Build Tools (SBT) for Eclipse.

Day 2: Familiarisation with the DE2-115 hardware, studying the user manual for the pin assignments of the LEDs, switches, and seven-segment displays. Created a simple project linking switches to LEDs in Verilog to verify the board and the programming cable.

Day 3: Began the first system-on-chip design in Platform Designer (Qsys). Instantiated a Nios II/e core, on-chip memory, and a JTAG UART, and assigned base addresses and interrupts.

Day 4: Generated the HDL from Qsys, instantiated the system in a top-level wrapper, assigned physical pins, compiled the project, and programmed the FPGA over the USBBlaster.

Day 5: Created a Board Support Package (BSP) from the generated .sopcinfo file in the Nios II SBT, wrote a C program to print to the console, and executed it on the soft processor.

Week 2: Hardware/Software Co-design

Day 6: Studied the Nios II custom-instruction interface and how the ALU pipeline can route operands through custom combinational logic.

Day 7: Designed a custom Verilog module for a bit-manipulation task that is inefficient in software, and verified its functional correctness by simulation before integration.

Day 8: Integrated the custom module into the Nios II through the Platform Designer custom-instruction interface, mapping the operand and result signals to meet the processor timing.

Day 9: Recompiled the hardware, regenerated the BSP, and located the C macros automatically generated for the custom instruction in the system headers.

Day 10: Wrote a C program that executed the task both in pure software and through the custom hardware macro, using an internal timer to measure the clock-cycle speedup.

Week 3: Memory and I/O Integration

Day 11: Added the SDRAM controller IP for the on-board 128 MB SDRAM in Qsys, configuring the CAS latency and address widths to match the memory datasheet.

Day 12: Instantiated a phase-locked loop (PLL) to generate a phase-shifted SDRAM clock so that data is captured correctly despite PCB routing delays.

Day 13: Tested the memory by writing and reading back data across the full SDRAM address range in C, verifying data integrity.

Day 14: Added general-purpose I/O (PIO) components connected to the LEDs and pushbuttons, and wrote an interactive memory-test program controlled from the buttons.

Day 15: Developed benchmarking code to measure memory bandwidth and latency, comparing the fast on-chip memory against the larger external SDRAM.

Week 4: Multi-core Architecture

Day 16: Began a new Qsys project and instantiated two Nios II cores (CPU_0 and CPU_1) to investigate multi-processor SoC architectures.

Day 17: Connected both CPUs to the shared SDRAM and added hardware mutex cores to manage access to shared memory and peripherals.

Day 18: Configured separate reset and exception vectors so that each core booted from its own region of memory without instruction overlap.

Day 19: Generated the dual-core hardware, compiled the project, and created a separate BSP and application for each core.

Day 20: Demonstrated true parallel execution: one core blinked red LEDs and printed to one terminal while the other blinked green LEDs and printed to a second terminal concurrently.

Week 5: RISC-V Implementation

Day 21: Pivoted to the open-source ecosystem and selected the NEORV32, a customisable 32-bit RISC-V soft core written in VHDL.

Day 22: Studied the NEORV32 top-level entity, set up a new Quartus project, and configured the VHDL library settings for correct compilation of the core.

Day 23: Wrote a top-level wrapper to instantiate the NEORV32 core, connecting its clock, reset, and UART signals to the physical pins of the DE2-115.

Day 24: Configured the RISC-V GCC toolchain and built the NEORV32 bootloader and core libraries, generating the memory-initialisation files.

Day 25: Synthesised the RISC-V core, programmed the FPGA, and executed C programs on the open-source architecture, establishing the build-and-run flow for the LDPC work.

Week 6: LDPC Decoder (Software and Demonstration)

Day 26: Began the core study of LDPC codes: parity-check matrices, Tanner graphs, and the min-sum decoding algorithm.

Day 27: Developed a floating-point reference model of the min-sum decoder in C, able to generate a codeword, add Gaussian noise, and perform iterative message passing.

Day 28: Validated the C model with Monte-Carlo simulations, plotting the bit-error-rate curves to confirm the algorithm corrects errors correctly.

Day 29: Ported the validated C decoder to run on the NEORV32 core on the FPGA, adapting it to the memory constraints of the embedded system.

Day 30: Built an interactive demonstration in which the decoder's progress and the status of the parity checks were displayed on the DE2-115 LEDs in real time.

Week 7: Characterisation and Literature Review

Day 31: Ran extended Monte-Carlo characterisation of the decoder, measuring the BER waterfall and the average number of iterations across a range of signal-to-noise ratios.

Day 32: Analysed the convergence behaviour, confirming that the decoder required several iterations at low SNR but corrected most frames almost immediately at high SNR.

Day 33: Compiled a literature review and annotated reading list tracing the development of LDPC codes from Gallager (1962) through the min-sum approximation and hardware implementations.

Day 34: Prepared a technical presentation on LDPC codes, the sum-product and min-sum algorithms, and the performance curves obtained from simulation.

Day 35: Presented the findings to the VeCAD research group and incorporated the feedback into the project plan for the hardware accelerator.

Week 8: CFU Design and Quantisation Study

Day 36: Profiled the embedded decoder and confirmed that the check-node min-sum operation dominated the run time, making it the natural target for hardware acceleration.

Day 37: Modified the C model to use fixed-point arithmetic and simulated word lengths of 3, 4, 5, and 6 bits for the log-likelihood ratios.

Day 38: Compared the fixed-point models against the floating-point reference, establishing that a 5-6 bit datapath matches the floating-point performance while a 3-bit datapath breaks down.

Day 39: Designed the check-node min-sum datapath in SystemVerilog, computing the first and second minima and the combined sign in parallel combinational logic.

Day 40: Verified the stand-alone CFU kernel by simulation against the software checknode function before integrating it into the processor.

Week 9: CFU Integration

Day 41: Enabled the NEORV32 Custom Functions Unit through the RISCV_ISA_Xcfu generic and connected the min-sum kernel to the CFU interface.

Day 42: Wrote the custom-instruction firmware, invoking the CFU from C through the RISC-V custom-0 opcode using the intrinsic instruction macro.

Day 43: Kept the message routing, variable-node updates, and syndrome check in software so that only the performance-critical kernel was offloaded to hardware.

Day 44: Verified that the accelerated decoder produced bit-identical results to the software decoder across a large set of test frames.

Day 45: Instrumented the firmware with the machine cycle counter (CSR mcycle) to measure the cycles spent in the check-node operation.

Week 10: Synthesis, Verification and Measurement

Day 46: Synthesised the complete CPU-plus-CFU system in Quartus Prime targeting the Cyclone IV E and inspected the compilation reports.

Day 47: Measured the cycle counts for the software and hardware versions of the checknode operation, obtaining an approximately five-fold speedup.

Day 48: Recorded the resource utilisation: the CFU added only 128 logic elements, bringing the complete system to 4,082 logic elements (about 4% of the device).

Day 49: Added a Synopsys Design Constraints (SDC) file to constrain the system clock and obtained a maximum operating frequency of 59.04 MHz from the Timing Analyzer.

Day 50: Verified correctness on the board through the LED indicators and confirmed that the accelerator introduced no loss of decoding accuracy.

Week 11: Documentation and Reporting

Day 51: Consolidated all measured results and prepared the figures and tables for the research paper and the final report.

Day 52: Wrote and formatted the IEEE-style research paper on the RISC-V min-sum LDPC accelerator.

Day 53: Prepared the UTM industrial training final report following the faculty template.

Day 54: Organised the Quartus project, the HDL source files, and the firmware, and wrote a short README for the archive.

Day 55: Reviewed the paper and report with the supervisor, incorporated the feedback, and handed over the documented project to the laboratory.

2.5 Skills and Knowledge Acquired

The industrial training developed a broad set of technical skills across the complete digital-design flow. On the hardware side, the trainee gained proficiency in Intel Quartus Prime and Platform Designer (Qsys), in writing and simulating registertransfer-level designs in Verilog, SystemVerilog, and VHDL, and in the physical implementation steps of synthesis, fitting, pin assignment, and timing closure using Synopsys Design Constraints. Bringing up both the proprietary Nios II and the opensource NEORV32 RISC-V soft processors, and extending the latter with a custom instruction, gave a thorough understanding of system-on-chip integration and hardware/software co-design.

On the algorithmic and software side, the trainee learned the theory and practice of channel coding, implementing a min-sum LDPC decoder in C, characterising it through Monte-Carlo simulation, and performing a fixed-point quantisation study to prepare the algorithm for hardware. Supporting skills included configuring the RISC-V GCC toolchain, using the machine performance counters for cycle-accurate measurement, and interpreting Quartus resource and timing reports.

Equally important were the professional skills developed throughout the training: systematic debugging of real hardware, independent problem solving when documentation was incomplete, planning and reporting work through weekly logbooks, and communicating technical results clearly in a presentation and a research paper. Working within the collaborative environment of the VeCAD laboratory also reinforced the value of peer discussion and continuous learning.

CHAPTER 3 SPECIAL TECHNICAL PROJECT

A RISC-V Custom-Instruction Accelerator for Fixed-Point Min-Sum LDPC Decoding on FPGA

3.1 Project Background

Low-density parity-check (LDPC) codes are a class of linear error-correcting codes defined by a sparse parity-check matrix H. Introduced by Gallager in 1962 and rediscovered in the late 1990s, they are capable of approaching the Shannon capacity of a noisy channel and are now adopted in modern standards such as Wi-Fi (802.11n/ac/ax), 5G New Radio, DVB-S2 satellite television, and 10GBASE-T Ethernet. LDPC codes are decoded iteratively on their Tanner graph using messagepassing (belief-propagation) algorithms, in which reliability information in the form of log-likelihood ratios (LLRs) is exchanged between variable nodes and check nodes until the decoder converges to a valid codeword or a maximum iteration count is reached.

The exact belief-propagation (sum-product) algorithm requires transcendental functions that are expensive to implement in hardware. The min-sum algorithm is a widely used simplification in which the check-node computation is approximated by the minimum of the incoming magnitudes multiplied by the product of their signs. This reduces the hardware cost dramatically while incurring only a small performance penalty, making min-sum the natural choice for an FPGA implementation.

When an LDPC decoder is executed in software on an embedded processor, the checknode min-sum operation dominates the run time because it is invoked many times per iteration and per decoded frame. This project investigates whether extending a RISCV soft core with a dedicated custom instruction for the check-node operation can

accelerate the decoder while keeping the hardware cost and the loss of error-correction performance negligible.

3.2 Objectives

The objectives of the special technical project were as follows:

1. To implement a fixed-point min-sum LDPC decoder in software on the opensource NEORV32 RISC-V soft core running on the DE2-115 FPGA board.
2. To study the effect of fixed-point quantisation of the log-likelihood ratios on decoding performance, and to determine the smallest word length that matches a 32-bit floating-point reference.
3. To design a custom functional unit (CFU) that performs the check-node minsum operation in hardware and to integrate it into the RISC-V datapath as a custom instruction.
4. To synthesise and verify the accelerator on the FPGA, and to measure the resulting speedup, logic-resource overhead, maximum operating frequency, and any change in decoding accuracy.

3.3 Theoretical Foundation

3.3.1 FPGA and Soft-Processors

Field-programmable gate arrays (FPGAs) are semiconductor devices based around a matrix of configurable logic blocks connected by programmable interconnect. Unlike application-specific integrated circuits (ASICs), which are manufactured for a fixed function, FPGAs can be reprogrammed after manufacture, making them ideal for rapid prototyping and hardware acceleration. The core resources of an FPGA are its logic elements — look-up tables and flip-flops that implement combinational and sequential logic — together with embedded memory blocks, digital-signal-processing multipliers, and high-speed I/O. The Cyclone IV E device (EP4CE115F29C7) on the DE2-115 board provides 114,480 logic elements, which is ample for complex SoC designs.

A soft processor is a microprocessor implemented entirely in the FPGA fabric rather than as fixed silicon. This offers flexibility — the instruction set, memory map, and peripherals can all be tailored to the application — at the cost of lower clock speed and higher resource use than a hard processor. Two soft processors were used during the training: the Intel Nios II, a proprietary 32-bit core with a mature toolchain, and the NEORV32, an open-source 32-bit RISC-V core written in portable VHDL. The RISCV instruction set is attractive because it is open and extensible, allowing user-defined custom instructions — the mechanism exploited by this project.

3.3.2 LDPC Codes and the Tanner Graph

Forward error correction adds redundancy to transmitted data so that a receiver can detect and correct errors without retransmission. LDPC codes are linear block codes whose defining feature is a sparse parity-check matrix H, containing mostly zeros and only a small number of ones. This sparsity is the key to efficient iterative decoding and allows LDPC codes to operate very close to the Shannon limit. The code used in this project is a rate-1/2, length-128 regular (3,6) LDPC code.

The decoding of LDPC codes is naturally expressed on a Tanner graph, a bipartite graph with two sets of nodes: variable nodes, representing the bits of the codeword, and check nodes, representing the parity-check equations. An edge connects a variable node to a check node when the corresponding entry of H is a one. Decoding proceeds by passing messages along these edges, iteratively refining the estimate of each bit.

3.3.3 Min-Sum Decoding and Log-Likelihood Ratios

Messages on the Tanner graph are expressed as log-likelihood ratios (LLRs), which encode both the most likely value of a bit and the confidence in that value. The most common hardware decoding algorithm is the min-sum algorithm, an approximation of the sum-product algorithm. In each check-node update, the outgoing message takes a

sign equal to the product of the signs of the other incoming messages and a magnitude equal to the minimum of their magnitudes. Finding a minimum and a parity of signs is computationally cheap and maps efficiently to combinational hardware, which is precisely why min-sum is favoured for FPGA implementation. After each iteration the decoder computes the syndrome; decoding stops when the syndrome is zero or when a maximum iteration count is reached.

3.3.4 Fixed-Point Quantisation

Software decoders typically use floating-point arithmetic, which is accurate but expensive to implement in silicon. Fixed-point quantisation represents the continuous LLR values with a finite number of bits. Determining the optimal datapath width — the smallest number of bits that maintains acceptable error-correction performance while minimising hardware area — is a crucial design step, and it was studied carefully in this project before committing to a hardware datapath.

3.4 Methodology

3.4.1 Software Reference Decoder

A floating-point min-sum decoder for the rate-1/2, length-128 regular (3,6) LDPC code was first implemented in C as a golden reference. The decoder initialises each variablenode LLR from the channel observation, then iterates between check-node and variable-node updates, computing the syndrome after each iteration. The C model also simulates an additive white Gaussian noise (AWGN) channel so that the bit error rate can be measured as a function of the signal-to-noise ratio. Figure 3.2 shows the reference decoder reporting its decoding metrics.

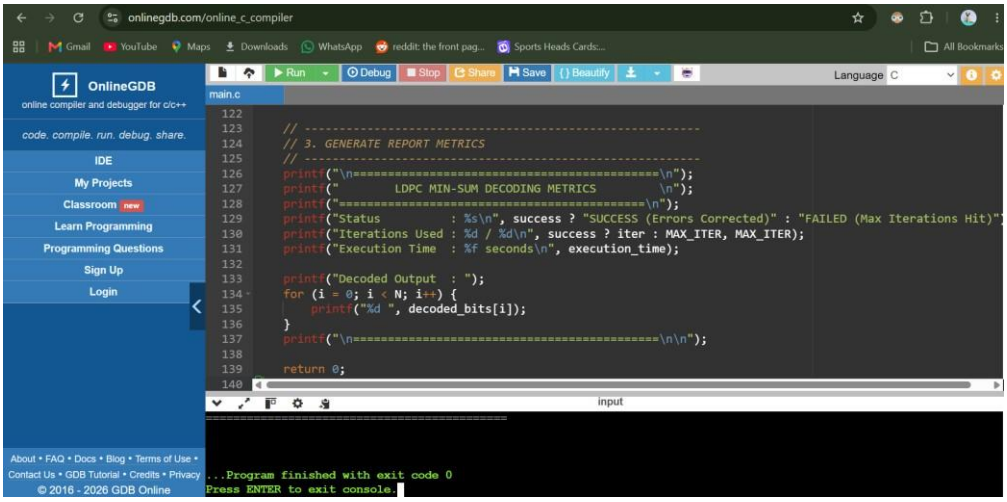


Figure 3.2 Software min-sum reference decoder reporting its decoding metrics.

3.4.2 Fixed-Point Quantisation Study

The floating-point model was modified to use fixed-point arithmetic, and the datapath width was varied over 3, 4, 5, and 6 bits. A Monte-Carlo simulation over the AWGN channel measured the BER as a function of Eb/N0 for each word length and compared the results against the 32-bit floating-point reference. The study established the minimum precision required before committing to a hardware datapath.

3.4.3 Custom-Instruction Accelerator Design

The NEORV32 exposes a Custom Functions Unit (CFU) that extends the instruction set with user-defined R-type instructions. The CFU was enabled through the processor's RISCV_ISA_Xcfu generic, and the check-node min-sum operation was implemented inside it in SystemVerilog. The custom datapath receives the packed incoming messages on the instruction operands, computes the first and second minima and the combined sign in parallel combinational logic, and returns the result to the register file in a small, fixed number of cycles. The top-level HDL design was captured in the Quartus text editor, as shown in Figure 3.1.

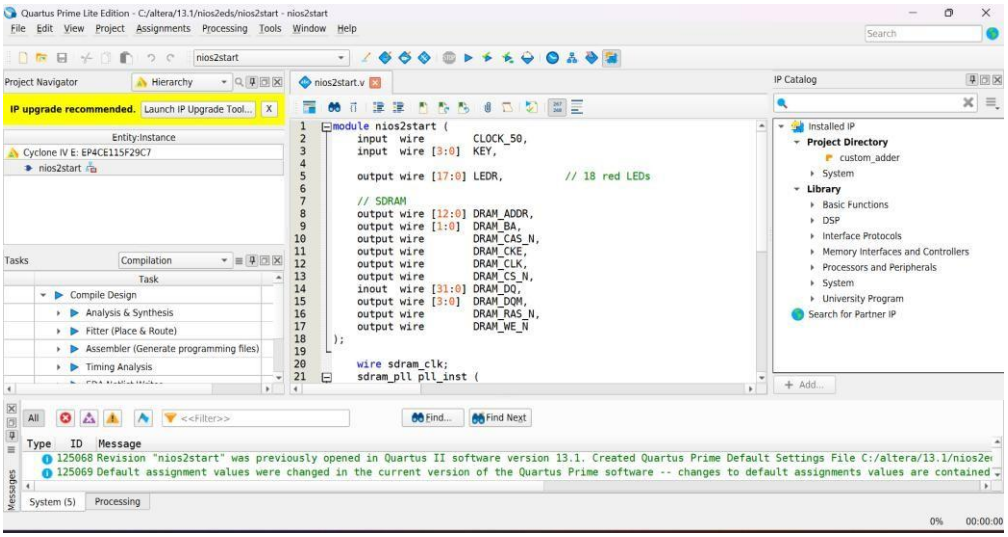


Figure 3.1 Top-level HDL design in the Quartus Prime text editor.

The custom instruction was invoked from C through the RISC-V custom-0 opcode using the intrinsic instruction macro provided by the NEORV32 software framework. The remainder of the decoder — message routing, variable-node updates, and the syndrome check — continued to run in software, so only the performance-critical kernel was offloaded to hardware, keeping the hardware small while preserving the flexibility of the software implementation.

3.4.4 System Integration and Compilation

The complete CPU-plus-CFU system was synthesised in Intel Quartus Prime targeting the Cyclone IV E (EP4CE115F29C7). The full compilation flow — analysis and synthesis, fitting (place-and-route), assembly, and timing analysis — is shown in Figure 3.3.

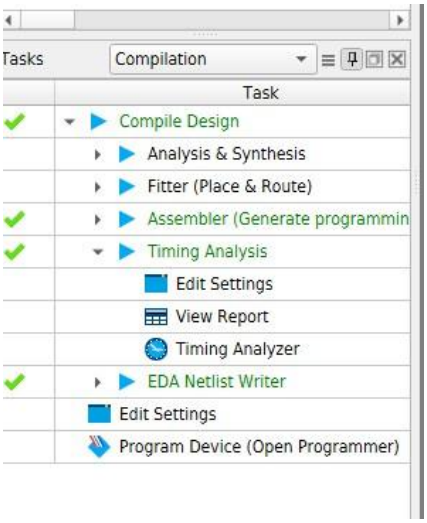


Figure 3.3 Quartus Prime compilation flow (Tasks pane) used to synthesise and verify the design.

3.4.5 Experimental Setup and Measurement

Cycle counts for the software and hardware versions of the check-node operation were obtained from the processor's machine cycle counter (CSR mcycle). Because no serial adapter was available, functional correctness and the relative speedup were displayed directly on the board's LEDs: a correct decode lit all indicators, after which the measured cycle counts were read out. A Synopsys Design Constraints (SDC) file was added to constrain the system clock so that the Timing Analyzer, shown in Figure 3.4, reported the maximum frequency of the user clock domain rather than the JTAG clock.

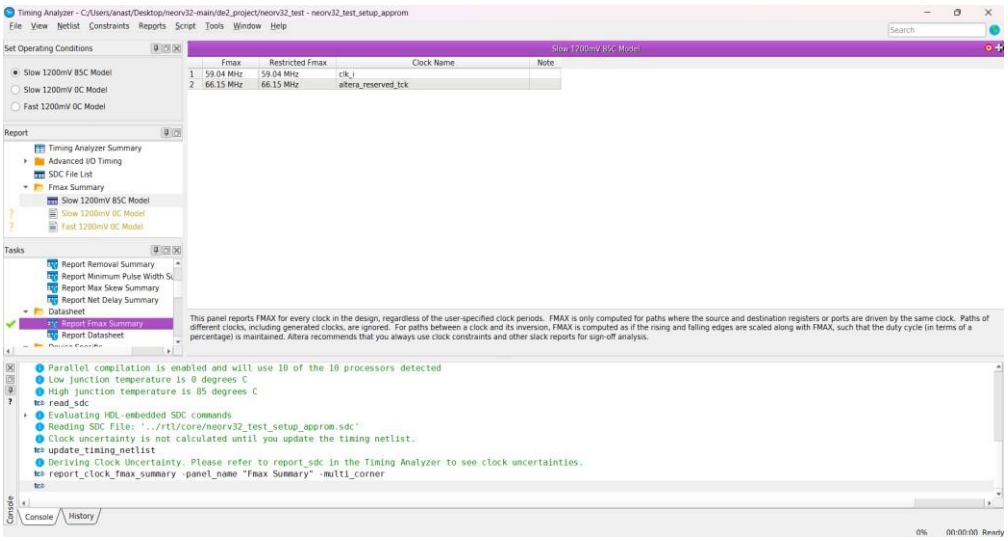


Figure 3.4 Quartus Timing Analyzer used to measure the maximum operating frequency.

3.5 Results and Discussion

3.5.1 Fixed-Point Quantisation

The fixed-point quantisation study confirmed that a 5-6 bit representation of the LLRs reproduces the error-correction performance of the 32-bit floating-point reference, while a 4-bit representation incurs a small loss of about 0.2 dB and a 3-bit representation degrades significantly. These results are summarised in Table 3.1, and the corresponding BER curves at higher and lower SNR are shown in Figures 3.5 and 3.6.

Table 3.1 Fixed-point quantisation versus 32-bit floating-point reference

Representation	BER vs float	Loss (dB)
32-bit floating point	Reference	0.00
6-bit fixed point	Matches reference	~0.00
5-bit fixed point	Matches reference	~0.05
4-bit fixed point	Slight degradation	~0.20
3-bit fixed point	Severe degradation	breaks down

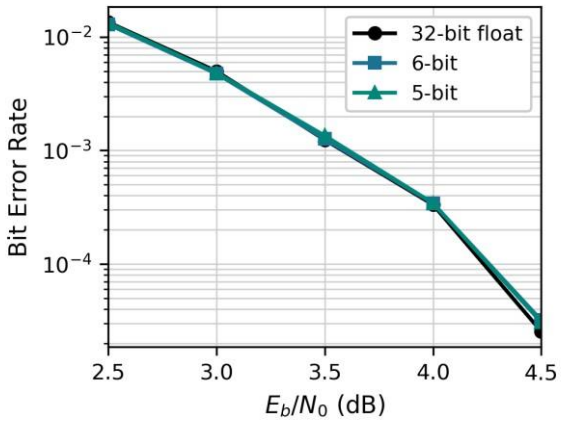


Figure 3.5 Bit-error-rate performance at higher SNR for the quantised decoders versus the floating-point reference.

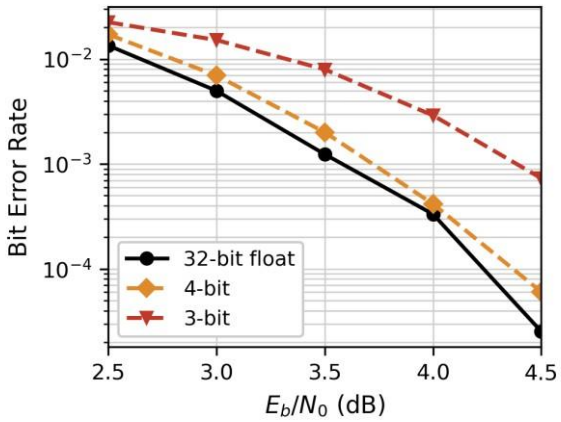


Figure 3.6 Bit-error-rate performance at lower SNR for the quantised decoders versus the floating-point reference.

3.5.2 Decoding Convergence

The average number of decoding iterations decreases with increasing SNR, as shown in Figure 3.7: at low SNR the decoder requires several iterations to converge, whereas at high SNR most frames are corrected almost immediately. This confirms the expected waterfall behaviour of the code and validates the decoder implementation.

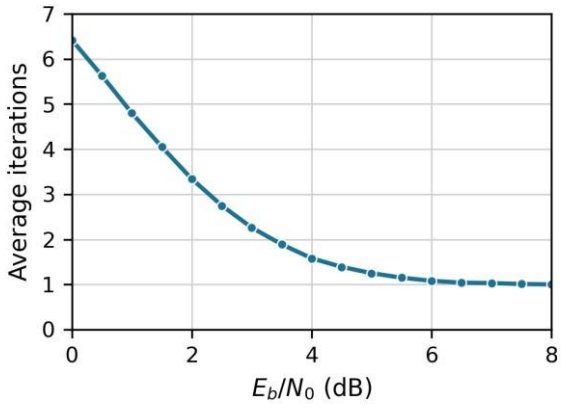


Figure 3.7 Average number of decoding iterations versus signal-to-noise ratio.

These results were obtained directly from the decoder running on the RISC-V core. Figure 3.8 reproduces the measured console output for the length-128, rate-1/2 code over 15,000 frames per point, confirming that the 6-bit and 5-bit fixed-point decoders

track the 32-bit floating-point reference in both bit error rate and average iteration count.

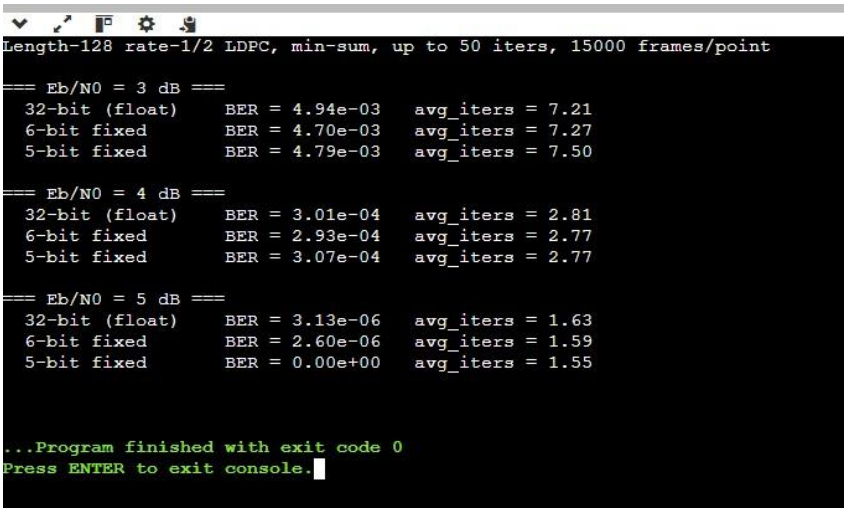


Figure 3.8 Measured bit error rate and average iterations for the floating-point and fixed-point decoders.

3.5.3 Hardware Acceleration

The custom-instruction accelerator produced a substantial reduction in decoding latency. Offloading the check-node min-sum operation to the CFU reduced the time spent in the dominant kernel from roughly 140 to about 26 clock cycles per check-node update — an approximately five-fold speedup — as summarised in Table 3.2 and illustrated in Figure 3.9.

Table 3.2 Decoder latency: software versus custom-instruction accelerator

Implementation	Cycles / check	Speedup
Software min-sum (RISC-V)	~140	1.0×
CFU custom instruction	~26	~5.4×

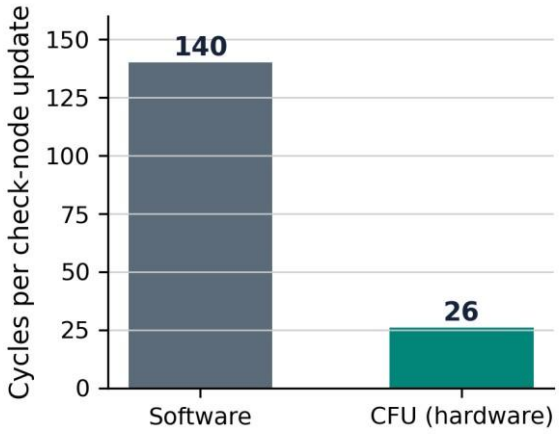


Figure 3.9 Clock cycles per check-node update: software versus CFU implementation.

3.5.4 Resource Utilisation and Timing

The additional hardware required for the accelerator was minimal. The CFU added only about 128 logic elements — roughly 0.1% of the Cyclone IV E device — and 27 registers, bringing the complete CPU-plus-CFU system to 4,082 logic elements (about 4% of the device). The design met timing with a maximum operating frequency of 59.04 MHz, comfortably above the 50 MHz system clock, and introduced no loss of decoding accuracy relative to the software baseline. These results are summarised in Table 3.3, and the Quartus Flow Summary is reproduced in Figure 3.10.

Table 3.3 FPGA resource utilisation and timing (Cyclone IV E)

Metric	Value
CFU logic elements	128 LE (~0.1%)
CFU registers	27
Full system logic elements	4,082 LE (~4%)
Maximum frequency (clk_i)	59.04 MHz
System clock	50 MHz (met)
Decoding accuracy vs software	No loss

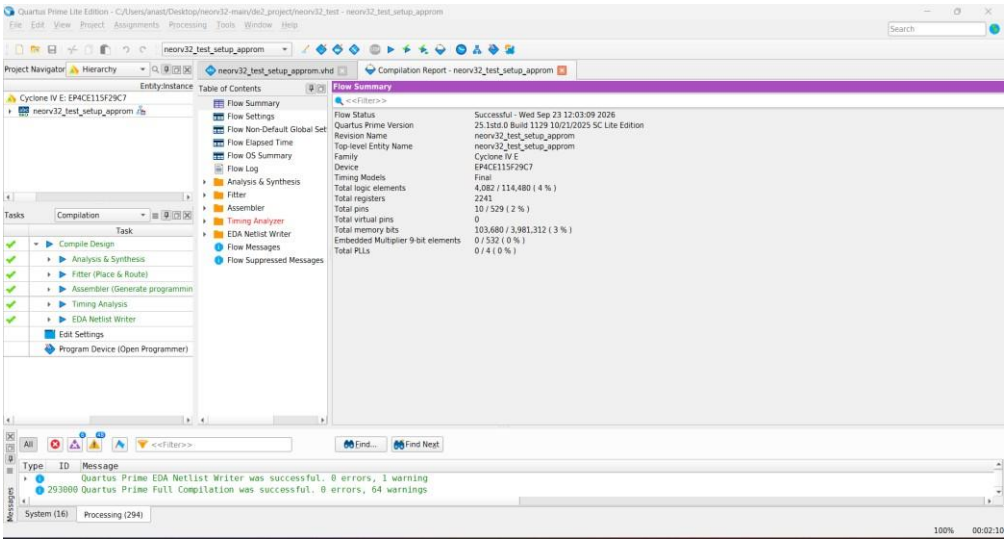


Figure 3.10 Quartus Flow Summary showing the measured resource utilisation of the CPU-plus-CFU system (4,082 logic elements on the Cyclone IV E).

3.5.5 On-Board Verification

Finally, the complete system was programmed onto the DE2-115 and the decoder was run on the board. A correct decode was confirmed through the on-board LED indicators, as shown in Figure 3.11, demonstrating the accelerator operating on real hardware.

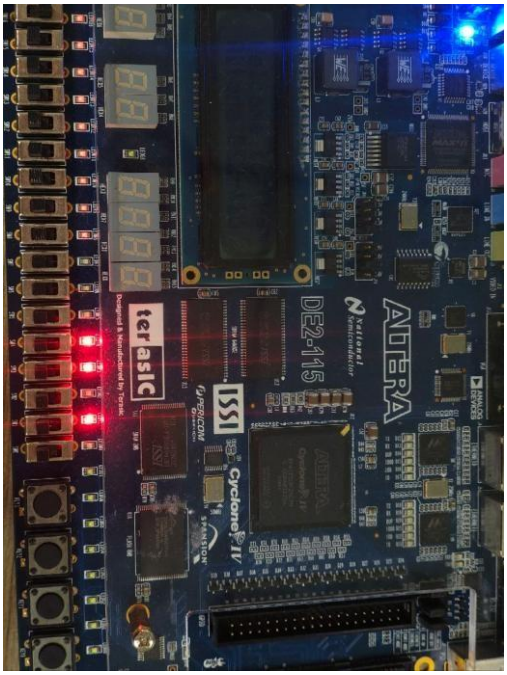


Figure 3.11 The LDPC decoder running on the DE2-115; the decoding result is displayed on the board LEDs.

3.5.6 Comparison and Discussion

Table 3.4 compares the accelerated decoder against the software baseline and a representative fixed-point min-sum implementation from the literature. The custominstruction accelerator preserves the error-correction performance of a 32-bit floatingpoint decoder using only a 5-6 bit datapath, while reducing the check-node latency by approximately five times at a negligible hardware cost. The low overhead means that the technique could be applied to larger codes or replicated for higher throughput without significantly affecting the resource budget of the host system.

Table 3.4 Comparison with the software baseline and a reference work

Design	Arithmetic	Datapath	Relative speed
Software (floating point)	32-bit float	n/a	1.0×
Software (fixed point)	integer	6-bit	~1.0×
This work (CFU)	integer	5-6 bit	~5.4×
Reference min-sum [7]	integer	5-6 bit	hardware

3.6 Conclusion

The special technical project successfully demonstrated that a small RISC-V custominstruction accelerator can significantly speed up min-sum LDPC decoding on an FPGA. The check-node kernel was accelerated by approximately five times at a cost of only 128 logic elements and with no loss of error-correction performance, while the fixed-point study showed that a 5-6 bit representation is sufficient to match a 32-bit floating-point decoder. The project provided practical experience of the complete FPGA design flow — from algorithm and HDL design through synthesis, timing closure, and on-board verification — and produced a reusable accelerator and a research paper. A natural extension of this work is the acceleration of quantum LDPC (qLDPC) decoding, an active direction for the laboratory.

CHAPTER 4 CONCLUSION

The industrial training at the VeCAD Research Laboratory provided a comprehensive and rewarding introduction to professional digital hardware design and research. Over the course of the training, the trainee progressed from first bringing up soft-core processors on the DE2-115 board to independently designing, implementing, and measuring a complete domain-specific hardware accelerator. The structured progression of weekly tasks ensured that each new skill — system integration, custominstruction design, memory and peripheral interfacing, multi-core design, and the opensource RISC-V flow — built upon the last and prepared the trainee for the final technical project.

The special technical project, a RISC-V custom-instruction accelerator for fixed-point min-sum LDPC decoding, met all of its objectives. It achieved an approximately fivefold acceleration of the check-node kernel at a negligible hardware cost of 128 logic elements, maintained a maximum operating frequency of 59.04 MHz, and preserved the decoding accuracy of the software baseline, while a fixed-point study established that a 5-6 bit word length matches a 32-bit floating-point reference. Beyond the technical results, the training strengthened the trainee's abilities in problem solving, independent debugging, technical writing, and the communication of engineering results.

Overall, the training achieved its purpose of bridging academic theory and engineering practice. The knowledge and skills gained — particularly in FPGA design, hardware/software co-design, and channel coding — form a strong foundation for future work, and the project lays the groundwork for the laboratory's continuing research into the hardware acceleration of advanced error-correcting codes, including quantum LDPC codes.

REFERENCES

[1] C. E. Shannon, “A mathematical theory of communication,” The Bell System Technical Journal, vol. 27, no. 3, pp. 379–423, 1948.

[2] R. G. Gallager, “Low-density parity-check codes,” IRE Transactions on Information Theory, vol. 8, no. 1, pp. 21–28, 1962.

[3] R. M. Tanner, “A recursive approach to low complexity codes,” IEEE Transactions on Information Theory, vol. 27, no. 5, pp. 533–547, 1981.

[4] D. J. C. MacKay and R. M. Neal, “Near Shannon limit performance of low density parity check codes,” Electronics Letters, vol. 32, no. 18, pp. 1645–1646, 1996.

[5] M. P. C. Fossorier, M. Mihaljević, and H. Imai, “Reduced complexity iterative decoding of low-density parity check codes based on belief propagation,” IEEE Transactions on Communications, vol. 47, no. 5, pp. 673–680, 1999.

[6] T. J. Richardson and R. L. Urbanke, “The capacity of low-density parity-check codes under message-passing decoding,” IEEE Transactions on Information Theory, vol. 47, no. 2, pp. 599–618, 2001.

[7] J. Zhao, F. Zarkeshvari, and A. H. Banihashemi, “On implementation of min-sum algorithm and its modifications for decoding low-density parity-check (LDPC) codes,” IEEE Transactions on Communications, vol. 53, no. 4, pp. 549–554, 2005.

[8] S. Nolting et al., “The NEORV32 RISC-V Processor,” open-source project documentation, 2024. [Online]. Available: https://github.com/stnolting/neorv32

[9] A. Waterman and K. Asanović, “The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA,” RISC-V Foundation, 2019.

[10] Terasic Inc., “DE2-115 Development and Education Board User Manual,” rev. 1.0, 2010.

[11] Intel Corporation, “Nios II Processor Reference Guide,” 2021.

[12] Intel Corporation, “Quartus Prime Standard Edition User Guide,” 2022.

APPENDIX

Appendix A — Weekly logbooks (Weeks 1–11), submitted separately.

Appendix B — Source files: ldpc_minsum.sv (check-node min-sum CFU kernel), neorv32_cpu_alu_cfu.vhd (CFU wrapper), neorv32_test_setup_approm.sdc (timing constraint), and main.c (decoder firmware).

Appendix C — IEEE Access research paper: “A RISC-V Custom-Instruction

Accelerator for Fixed-Point Min-Sum LDPC Decoding on FPGA."